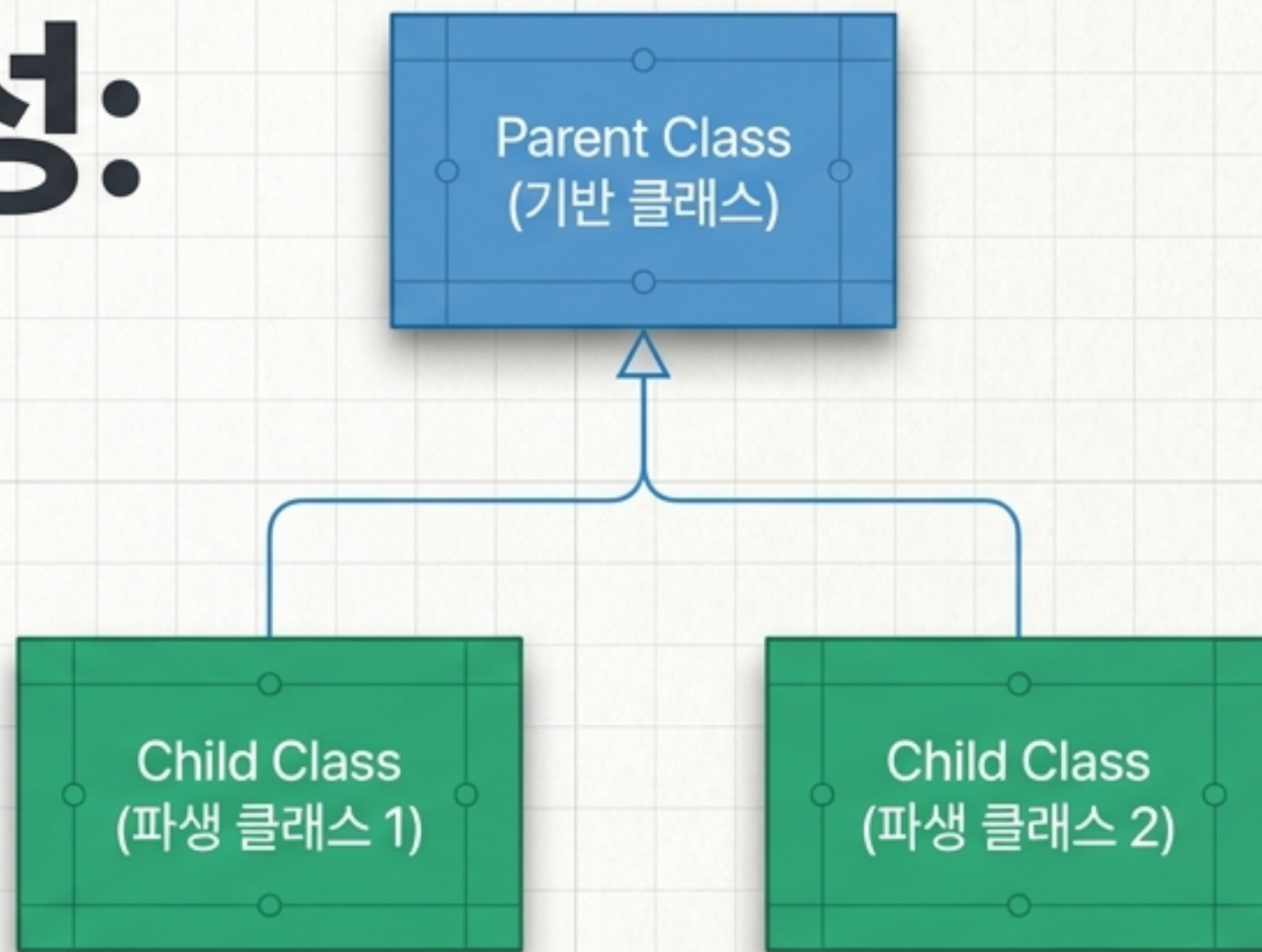


C# 객체지향의 완성: 상속과 다형성

코드의 중복을 없애고
유연한 아키텍처를 설계하는 법



복붙(Ctrl+C, V)의 저주

강아지와 고양이 클래스. 완전히 똑같은 속성과 행동(Age, Eat, Sleep)이 무의미하게 반복됩니다. 유지보수의 악몽이 시작되는 지점입니다.

Dog 클래스

```
01 class Dog
02 {
03     public int Age { get; set; }
04     public string Color { get; set; }
05
06     public Dog() { this.Age = 0; }
07
08     public void Eat() { Console.WriteLine("냠냠 먹습니다."); }
09     public void Sleep() { Console.WriteLine("쿨쿨 잠을 잡니다."); }
10     public void Bark() { Console.WriteLine("왈왈 짖습니다."); }
11 }
```

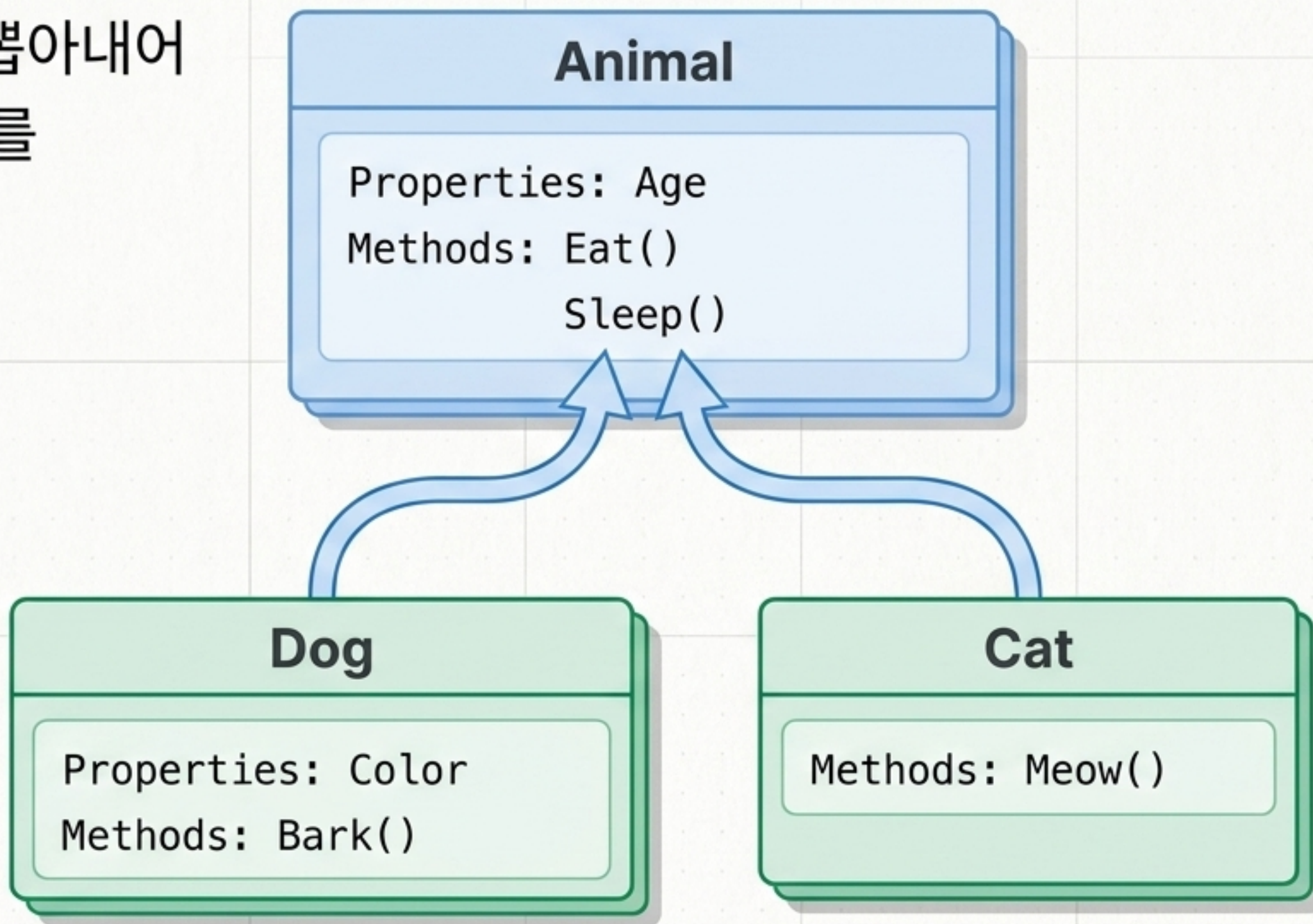


Cat 클래스

```
01 class Cat
02 {
03     public int Age { get; set; }
04
05     public Cat() { this.Age = 0; }
06
07     public void Eat() { Console.WriteLine("냠냠 먹습니다."); }
08     public void Sleep() { Console.WriteLine("쿨쿨 잠을 잡니다."); }
09     public void Meow() { Console.WriteLine("냥냥 읊니다."); }
10 }
```

설계도의 추출: 부모 클래스의 탄생

중복되는 속성과 메서드를 위로 뽑아내어
Animal이라는 공통 부모 클래스를
설계합니다.



코드 다이어트: 상속의 실제 구현

콜론(:) 기호 하나로 부모의 모든 능력을 물려받습니다.
코드는 가벼워지고, 핵심 정체성만 남습니다.

class Dog : Animal

```
01 class Dog : Animal
02 {
03     public string Color { get; set; }
04     public void Bark() { Console.WriteLine("알알 짹습니다."); }
05 }
06 class Cat : Animal
07 {
08     public void Meow() { Console.WriteLine("냥냥 읊니다."); }
09 }
```

물려받은
코드는 생략됨
(핵심 기능만
유지)

권한 진단 매트릭스: 접근 제한자

protected는 부모와 자식 간에만 허락된 가보(가족 비밀)와 같습니다.
외부 시스템에서는 철저히 감춰집니다.

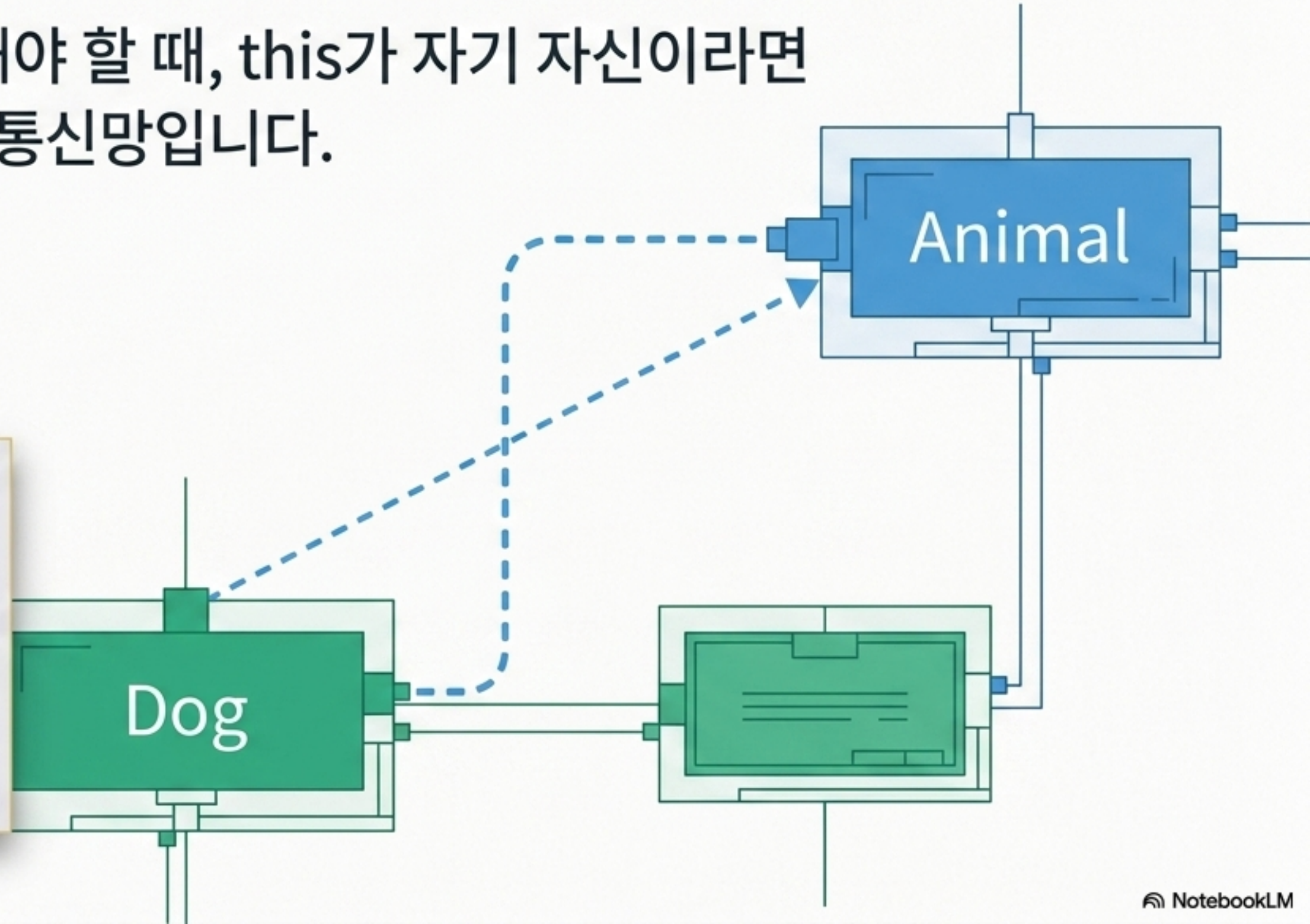
	내부 클래스 (Inside)	파생 클래스 (Child)	외부 클래스 (Outside)
public	✓	✓	✓
protected	✓	✓ 	✗ 
private	✓	✗	✗

부모와 연결되는 직통 인터폰: base 키워드

부모의 메서드에 강제로 접근해야 할 때, this가 자기 자신이라면 base는 부모를 호출하는 직통 통신망입니다.

```
class Dog : Animal {  
    ...  
    base.  
}
```

- Age
- Eat
- Equals
- GetHashCode
- GetType
- MemberwiseClone
- Sleep
- ToString



타입의 장벽: 100종의 동물이 있다면?

동물의 종류가 늘어날 때마다 새로운 리스트와 새로운 반복문이 끝없이 추가됩니다.
비효율의 늪에 빠집니다.

```
List<Dog> Dogs = new List<Dog>();  
List<Dog> Dogs = new List<Dog>();  
List<Cat> Cats = new List<Cat>();  
List<Snake> Lizards = new List<Snake>();  
List<Snake> Lizards = new List<Snake>();  
List<Tiger> Elepents = new List<Tiger>();  
List<Tiger> Elepents = new List<Tiger>();  
List<Lion> Lions = new List<Lion>();  
List<Lion> Lions = new List<Lion>();  
List<Lion> Lions = new List<Lion>();
```

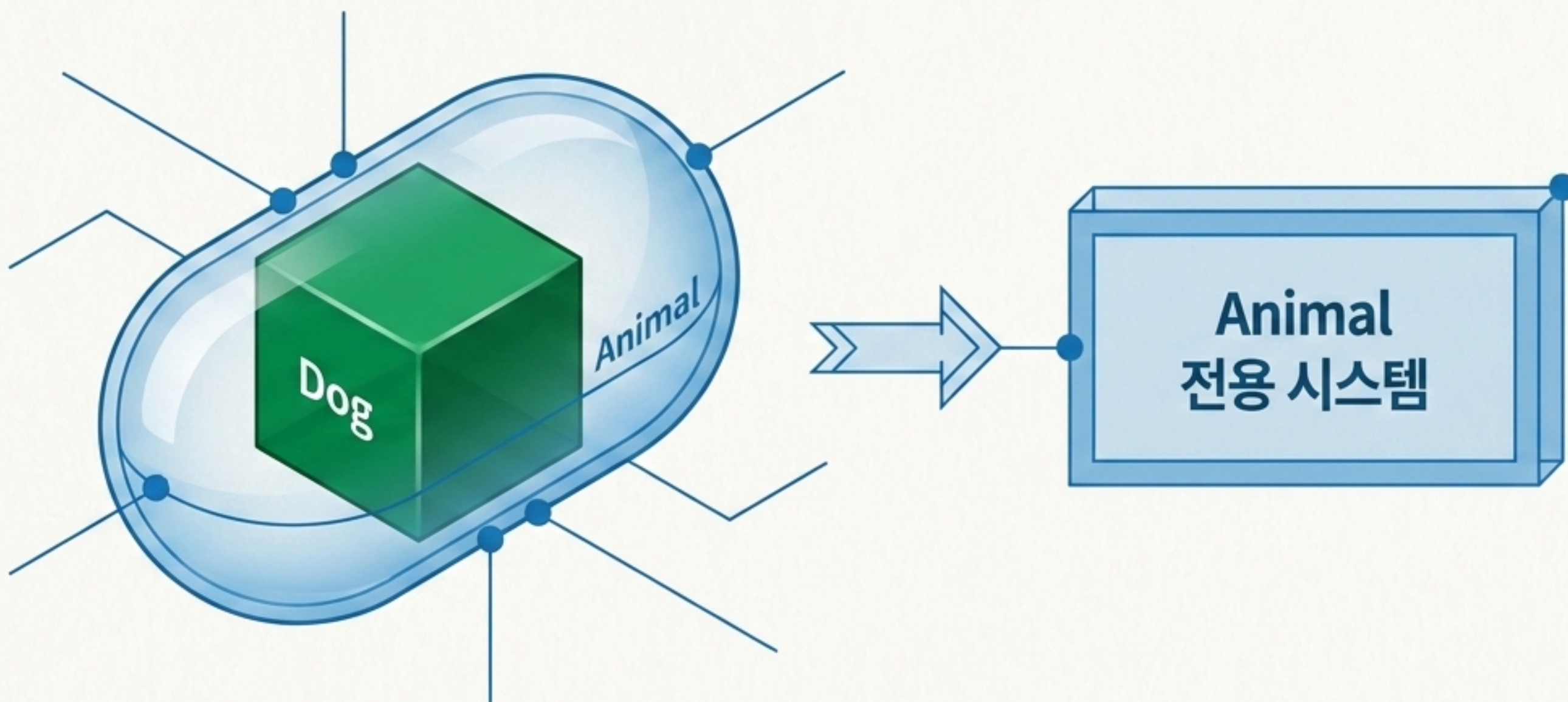
코드 중복 발생 위험
(유지보수 불가)

완벽한 위장술: 다형성(Polymorphism)

```
Animal dog = new Dog();
```

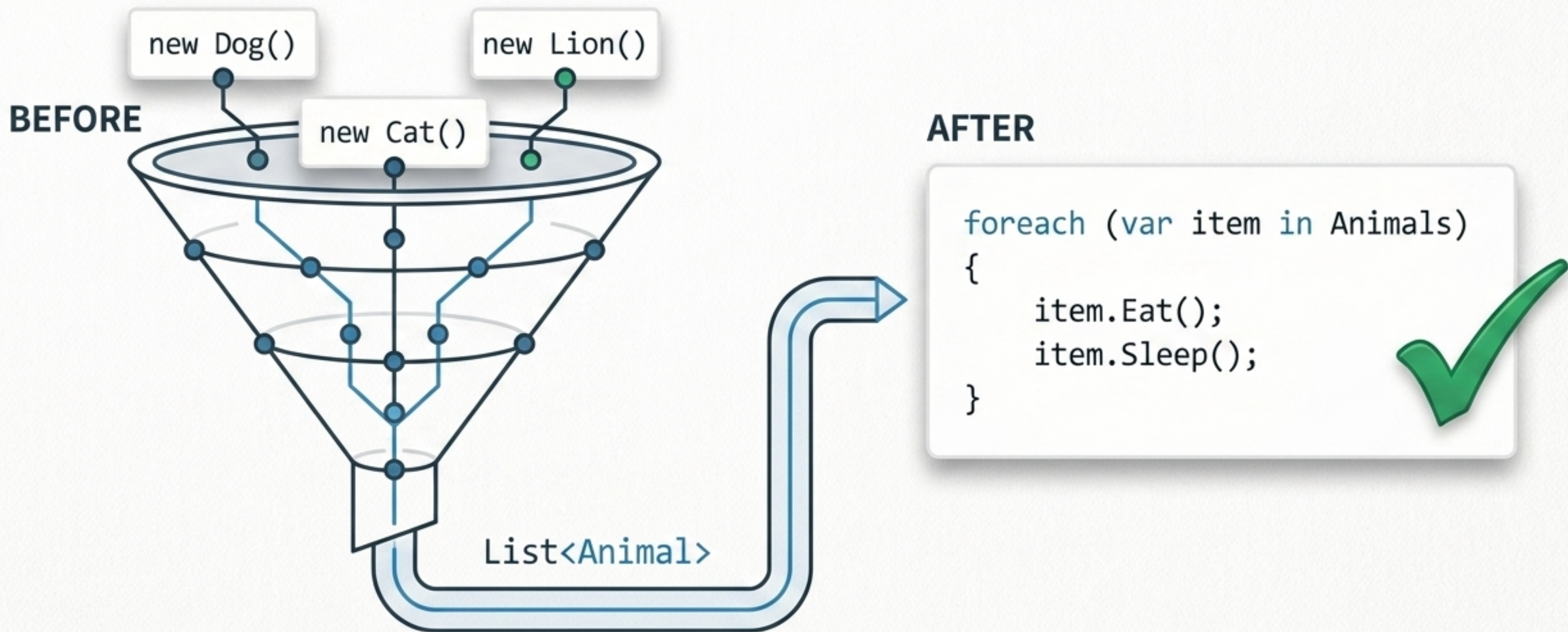
본질은 Dog이지만 겉옷은 Animal을 입습니다.

하위 타입은 언제나 상위 타입으로 변환될 수 있습니다.



단 하나의 리스트 (One List to Rule Them All)

다형성 덕분에 흩어져 있던 객체들이 `List<Animal>` 하나로 통합됩니다.
단 하나의 `foreach` 루프로 모든 동물을 제어합니다.



위장의 대가: 고유 기능의 제한

Animal로 위장한 상태에서는 Animal의 기능(Eat, Sleep)만. Dog 고유의 기능(BinBark)은 시야 밖으로 가려집니다.



```
Animal dog = new Dog();  
dog.
```

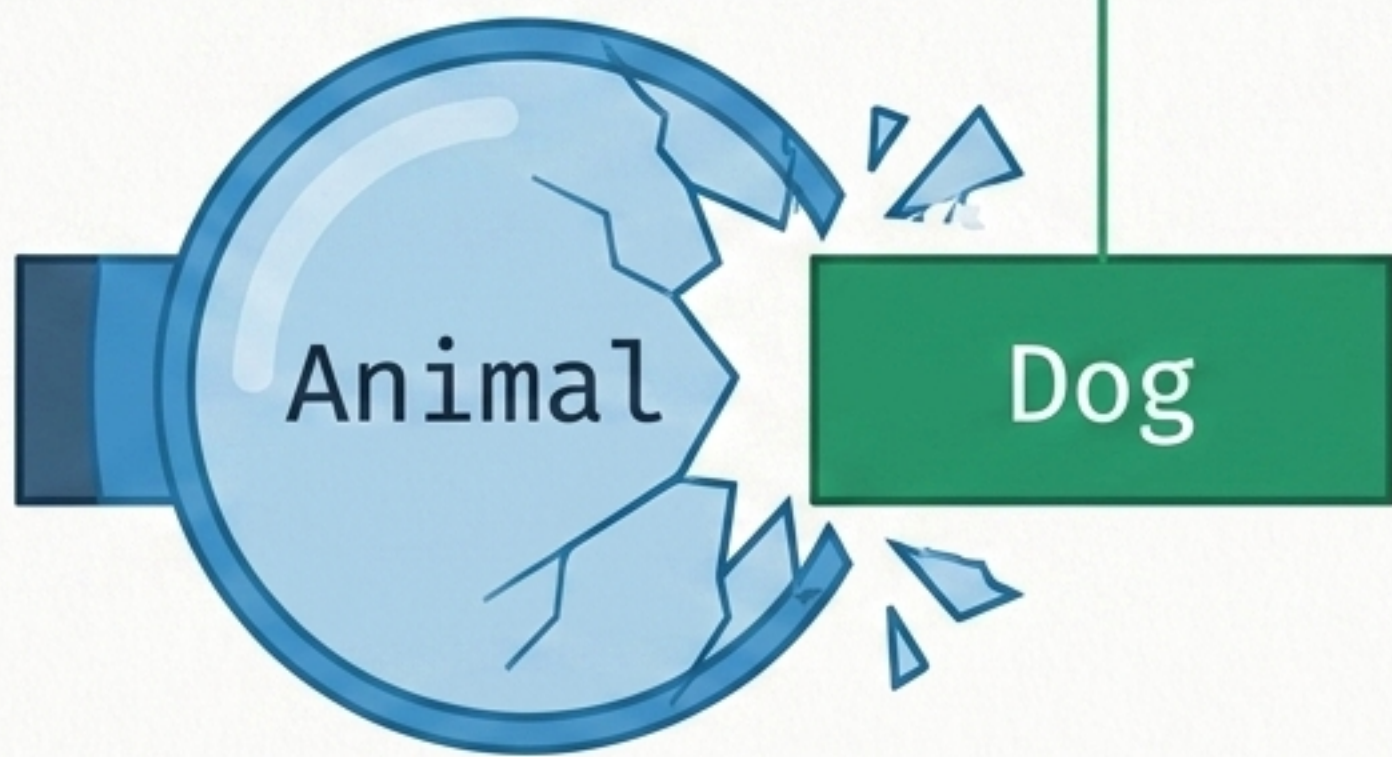
- Age
- Equals
- Sleep
- ToString
- GetType
- Age
- Eat
- Equals
- GetHashCode

이 컨텍스트를 기반으로 한 IntelliCode 제안

위장 해제: 캐스팅(Casting)

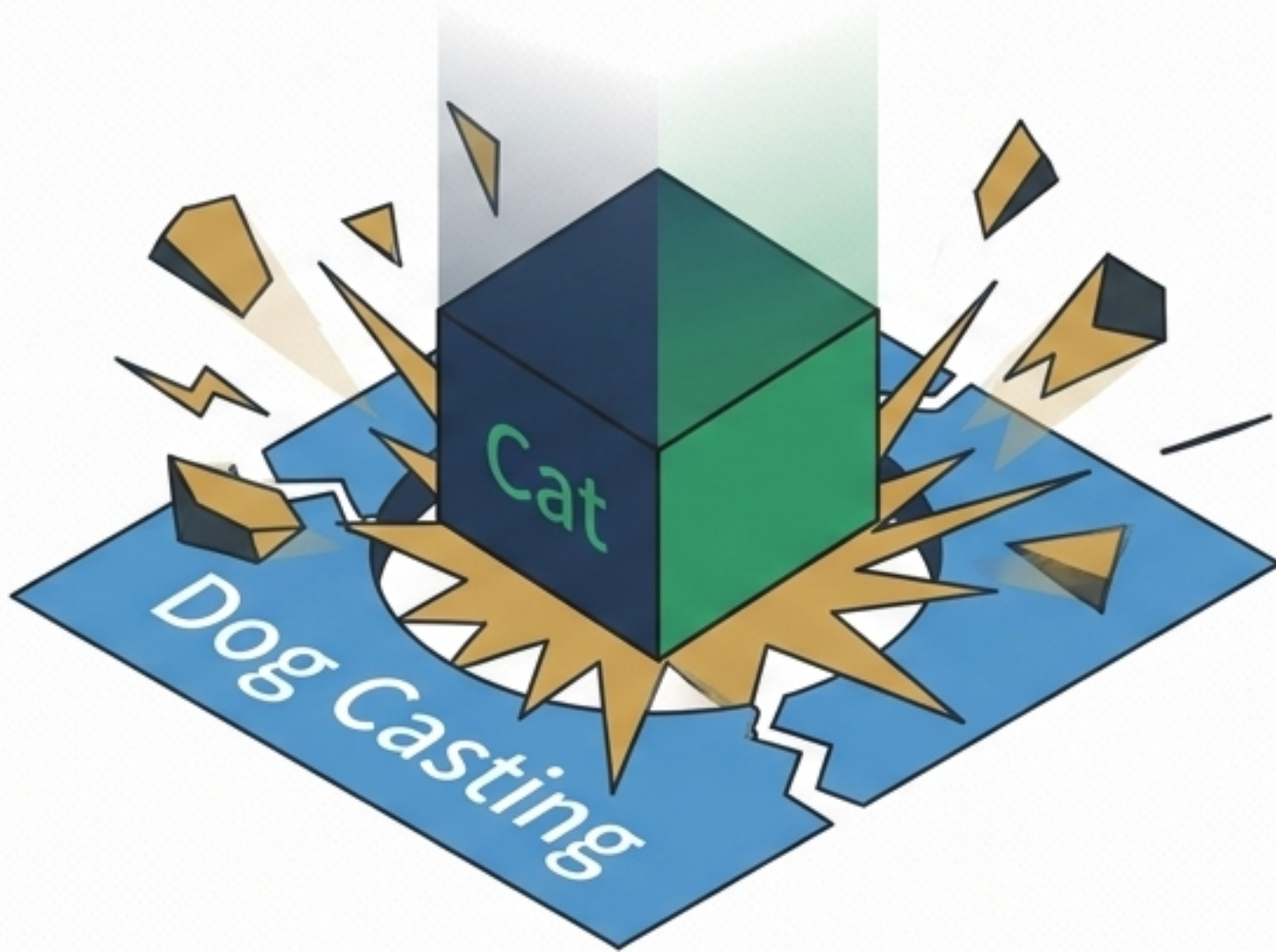
가려진 고유 능력을 다시 사용하려면, 원래의 모습으로 강제 변환(캐스팅)해야 합니다.

```
((Dog)item).Bark();
```



위험한 가정: 무작정 캐스팅의 대가

껍질 속에 Cat이 들어있는데 Dog로 억지로 위장을 벗기려 한다면?
즉각적인 런타임 에러가 발생하며 프로그램이 붕괴합니다.



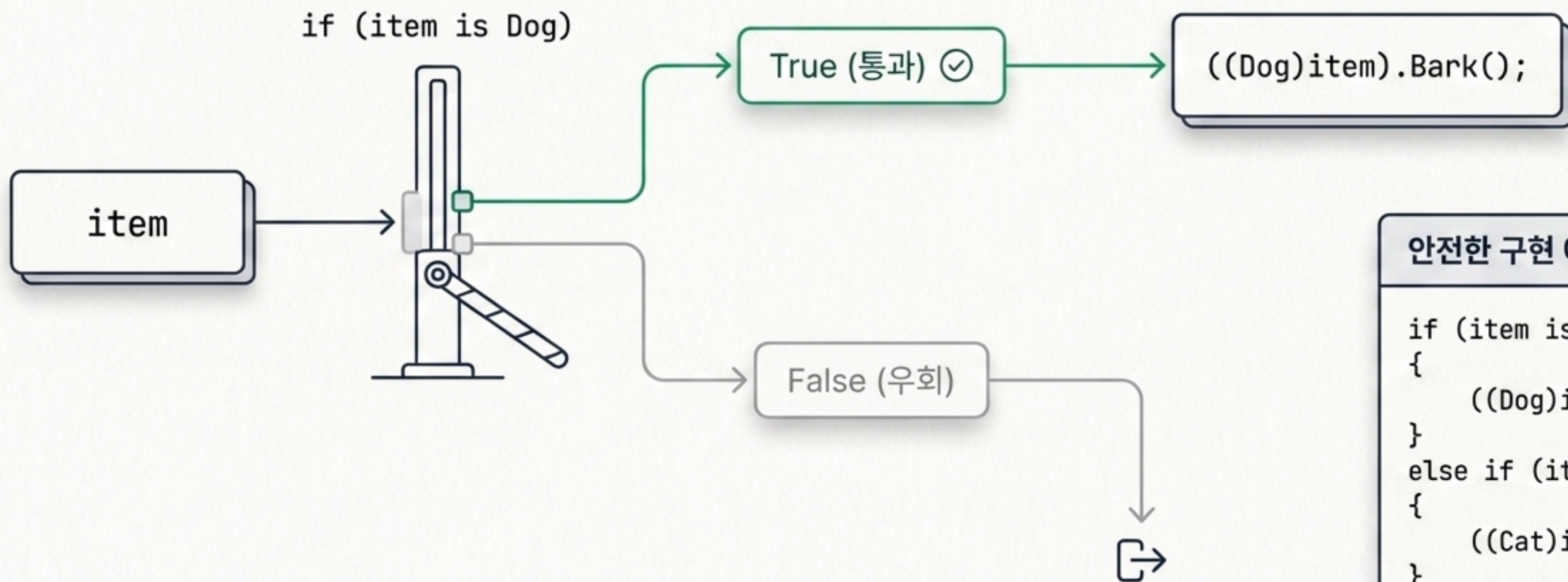
```
01 foreach (var item in Animals)
02 {
03     item.Eat();
04     item.Sleep();
05     ((Cat)item).Meow();
06 }
```

처리되지 않은 예외: System.InvalidCastException:
'Dog' 형식 개체를 'Cat' 형식으로 캐스팅할 수 없습니다.

안전 검문소: is 키워드

캐스팅 전, 객체의 진짜 신분증을 확인합니다.

is 연산자를 통과한 객체만 안전하게 고유 메서드를 실행합니다.

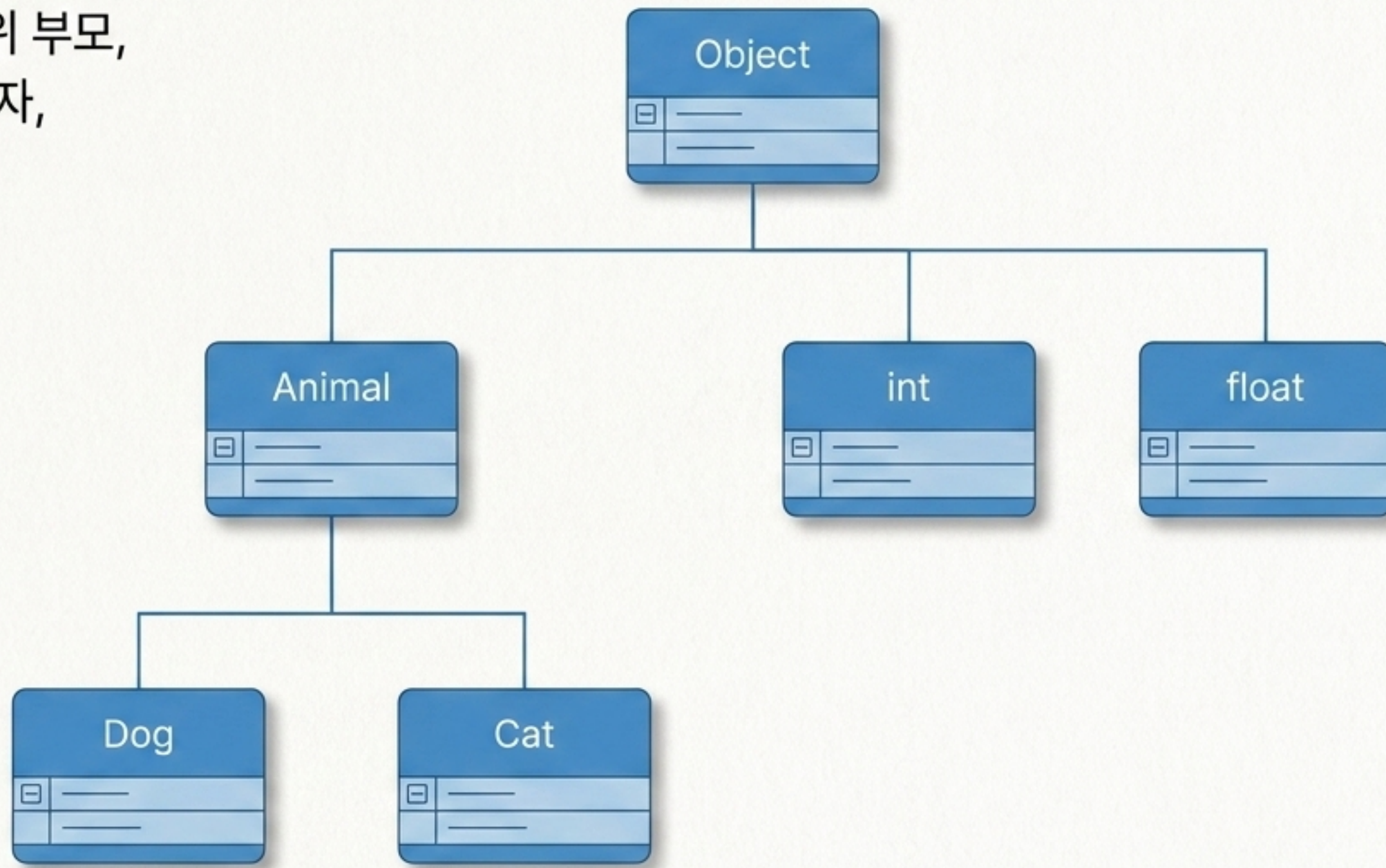


안전한 구현 예시

```
if (item is Dog)
{
    ((Dog)item).Bark();
}
else if (item is Cat)
{
    ((Cat)item).Meow();
}
```

궁극의 조상: 최상위 Object 클래스

C#의 모든 클래스는 보이지 않는 최상위 부모, **Object**를 상속받습니다. 숫자(int), 문자, 사용자 정의 클래스까지 모두 거대한 하나의 가계도로 연결됩니다.



진화하는 아키텍처: 마스터 맵

코드의 중복을 제거하고(상속), 데이터의 관리를 하나로 묶어내며(다형성),
안전하게 개별 기능에 접근하는(캐스팅과 is) 객체지향의 완벽한 흐름입니다.

1. 상속 (Inheritance)



코드 재사용 및 구조화

2. 다형성 (Polymorphism)



단일 인터페이스 통합
(List<Animal>)

3. 안전한 캐스팅 (Casting & is)



타입 검증 및
고유 기능 복원